

Marcar como hecha

Facade

Es un patrón de diseño estructural que proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases.

El objetivo del patrón de diseño **Facade** es agrupar las interfaces de un conjunto de objetos en una interfaz unificada volviendo a este conjunto más fácil de usar.

Imagina que debes lograr que tu código trabaje con un amplio grupo de objetos que pertenecen a una sofisticada biblioteca o *framework*.

Una fachada es una clase que proporciona una interfaz simple a un subsistema complejo que contiene muchas partes móviles

Tener una fachada resulta útil cuando tienes que integrar tu aplicación con una biblioteca sofisticada con decenas de funciones, de la cual sólo necesitas una pequeña parte

Puedes en él, inicializar objetos en el orden correcto y suministrarles datos en el formato adecuado.

Las clases del subsistema no conocen la existencia de la fachada. Operan dentro del sistema y trabajan entre sí directamente.

El **Cliente** utiliza la fachada en lugar de invocar directamente los objetos del subsistema.

Regresemos a la idea de la tienda de coches online.

La arquitectura del sistema está formada por un conjunto de componentes donde cada uno posee su propia interfaz como:

- el componente Catalogo,
- el componente GestionDocumento.

Es posible dar acceso al conjunto de la interfaz de estos componentes a los clientes del servicio web, aunque esta posibilidad presenta dos inconvenientes principales:

- Algunas funcionalidades no las utilizan los clientes del servicio web, como por ejemplo las funcionalidades de visualización del catálogo.
- La arquitectura interna del sistema responde a las exigencias de modularidad y evolución que no forman parte de las necesidades de los clientes del servicio web.

El patrón de diseño Facade resuelve este problema proporcionando una interfaz unificada más sencilla y con un nivel de abstracción más elevado.

Conviene observar que si bien la idea del patrón de diseño es construir una interfaz de más alto nivel de abstracción, nada nos impide proporcionar en la fachada accesos directos a ciertos métodos de los componentes del sistema.

La fachada no puede limitarse a transmitir las llamadas. También debe realizar la adaptación entre su interfaz y la interfaz de los objetos del sistema mediante código específico.

El patrón de diseño **Facade** utiliza en los siguientes casos:

- Para proveer una interfaz simplificada de un sistema complejo. La arquitectura de un sistema puede estar basada en numerosas clases pequeñas, que ofrecen una buena modularidad y capacidad de evolución. No obstante estas propiedades tan estupendas no interesan en absoluto a los clientes, que solo necesitan un acceso simple que responda a sus exigencias.
- Para dividir un sistema en subsistemas, la comunicación entre subsistemas se define gracias a las fachadas.
- Para sistematizar la encapsulación de la implementación de un sistema de cara al exterior.

?

Ejemplo de código

```
<?php

interface CatalogoInterfaz
{
    public function encuentraVehiculos(float $precioMin, float
$precioMax): array;
}
<?php

    include('CatalogoInterfaz.php');

class ComponenteCatalogo implements CatalogoInterfaz
{
    protected array $vehiculos;

    public function __construct(array $vehiculos)
    {
        $this->vehiculos = $vehiculos;
    }

    public function encuentraVehiculos(float $precioMin, float
$precioMax): array
    {
        $resultado = [];

        foreach ($this->vehiculos as $vehiculo) {
            $precio = $vehiculo->getPrecio();

            if (($precio >= $precioMin) && ($precio <= $precioMax)) {
                $resultado[] = $vehiculo->getDescripcion() . '
(precio: ' . $precio . ')';
            }
        }

        return $resultado;
    }
}

<?php

class Vehiculo
{
    protected string $descripcion;

    protected float $precio;

    public function __construct(string $descripcion, float $precio)
    {
        $this->descripcion = $descripcion;
        $this->precio = $precio;
    }

    public function getDescripcion(): string
    {
        return $this->descripcion;
    }

    public function getPrecio(): float
    {
        return $this->precio;
    }
}
}
```

?

Continuamos con el componente de gestión de documentos constituido por la interfaz GestionDocumentoInterfaz y la clase

ComponenteGestionDocumento. Este componente constituye la simulación de una base de documentos.

```
<?php

interface GestionDocumentoInterfaz
{
    public function documento(int $indice): string;
}

<?php
include('GestionDocumentoInterfaz.php');

class ComponenteGestionDocumento implements
GestionDocumentoInterfaz
{
    public function documento(int $indice): string
    {
        return "Documento número $indice de la base documental" . "<br>";
    }
}
```

La interfaz de la fachada llamada WebServiceAutoInterfaz contiene la firma de dos métodos destinados a los clientes del servicio web.

```
<?php

interface WebServiceAutoInterfaz
{
    public function buscaDocumento(int $indice): string;

    public function buscaVehiculos(float $precioMedio, int
$desviacionMax): array;
}
```

La clase WebServiceAuto implementa ambos métodos. Destacamos el cálculo del precio mínimo y máximo para poder invocar al método encuentraVehiculos de la clase ComponenteCatalogo.

```
<?php

include('WebServiceAutoInterfaz.php');

class WebServiceAuto implements WebServiceAutoInterfaz
{
    protected CatalogoInterfaz $catalogo;

    protected GestionDocumentoInterfaz $gestionDocumento;

    public function __construct(CatalogoInterfaz $catalogo,
GestionDocumentoInterfaz $gestionDocumento)
    {
        $this->catalogo = $catalogo;
        $this->gestionDocumento = $gestionDocumento;
    }

    public function buscaDocumento(int $indice): string
    {
        return $this->gestionDocumento->documento($indice);
    }

    public function buscaVehiculos(float $precioMedio, int
$desviacionMax): array
    {
        return $this->catalogo->encuentraVehiculos($precioMedio -
$desviacionMax, $precioMedio + $desviacionMax);
    }
}
```

Por último, un cliente del servicio web puede escribirse en PHP como sigue:

?

```
<?php

// include('CatalogoInterfaz.php');
include('ComponenteCatalogo.php');
include('ComponenteGestionDocumento.php');
// include('GestionDocumentoInterfaz.php');
include('Vehiculo.php');
include('WebServiceAuto.php');
// include('WebServiceAutoInterfaz.php');

$vehiculos = [
    new Vehiculo('Berlina 5 puertas', 6999.99),
    new Vehiculo('Compacto 3 puertas', 4000),
    new Vehiculo('Espace 5 puertas', 8000),
    new Vehiculo('Break 5 puertas', 7000),
    new Vehiculo('Coupé 2 puertas', 9000),
    new Vehiculo('Utilitario 3 puertas', 5000),
];

$webServiceAuto = new WebServiceAuto(new ComponenteCatalogo($vehiculos), new ComponenteGestionDocumento());
echo "<h3>->buscaDocumento(1)</h3>";
echo $webServiceAuto->buscaDocumento(1);
echo "<h3>->buscaDocumento(2)</h3>";
echo $webServiceAuto->buscaDocumento(2);

$precioMedio = 6000;
$desviacionMax = 1000;
echo "<h3>->buscaVehiculos()</h3>";
$resultados = $webServiceAuto->buscaVehiculos($precioMedio, $desviacionMax);

if (($numVehiculos = count($resultados)) > 0) {
    $precioMin = $precioMedio - $desviacionMax;
    $precioMax = $precioMedio + $desviacionMax;

    echo $numVehiculos . " vehículo(s) tienen un precio comprendido entre $precioMin y $precioMax" . "<br>";

    foreach ($resultados as $resultado) {
        echo $resultado . "<br>";
    }
}
```

Este cliente muestra dos documentos así como los vehículos cuyo precio está comprendido entre 5.000 y 7.000. El resultado de la ejecución de este programa es el siguiente:

->buscaDocumento(1)

Documento número 1 de la base documental

->buscaDocumento(2)

Documento número 2 de la base documental

->buscaVehiculos()

3 vehículo(s) tienen un precio comprendido entre 5000 y 7000

Berlina 5 puertas (precio: 6999.99)

Break 5 puertas (precio: 7000)

Utilitario 3 puertas (precio: 5000)

Ejemplo segundo de código

```

<?php
/**
 * La clase Facade proporciona una interfaz sencilla a la compleja lógica de uno o
 * varios subsistemas. La Fachada delega las peticiones del cliente a los
 * objetos apropiados dentro del subsistema. La Fachada también es responsable de
 * gestionar su ciclo de vida. Todo esto protege al cliente de la indeseada
 * complejidad del subsistema.
 */
class Facade
{
    protected $subsystem1;
    protected $subsystem2;
    /**
     * Dependiendo de las necesidades de tu aplicación, puedes proporcionar a la Fachada
     * objetos de subsistema existentes o forzar a la Fachada a crearlos por sí misma.
     */
    public function __construct(
        Subsystem1 $subsystem1 = null,
        Subsystem2 $subsystem2 = null
    ) {
        $this->subsystem1 = $subsystem1 ? : new Subsystem1();
        $this->subsystem2 = $subsystem2 ? : new Subsystem2();
    }
    /**
     * Los métodos de la Fachada son cómodos accesos directos a la sofisticada
     * funcionalidad de los subsistemas. Sin embargo, los clientes sólo llegan a una fracción
     * Dependiendo de las necesidades de tu aplicación, puedes proporcionar a la Fachada los objetos del subsistema existentes
     o forzar a la Fachada a crearlos por sí misma.
     * objetos de subsistema existentes o forzar a la Fachada a crearlos por sí misma.
     */
    public function operation(): string
    {
        $result = "Facade inicializa los subsistemas:<br>";
        $result .= $this->subsystem1->operation1();
        $result .= $this->subsystem2->operation1();
        $result .= "Facade ordena a los subsistemas que realicen la acción:<br>";
        $result .= $this->subsystem1->operationN();
        $result .= $this->subsystem2->operationZ();

        return $result;
    }
}

/**
 * El subsistema puede aceptar peticiones tanto de la fachada como del cliente directamente.
 * En cualquier caso, para el Subsistema, la Fachada es un cliente más, y no es
 * una parte del Subsistema.
 */
class Subsystem1
{
    public function operation1(): string
    {
        return "Subsystem1: Listo!<br>";
    }
    public function operationN(): string

```

?

```
{
    return "Subsystema1: Vamos!<br>";
}

}

/**
 * Algunas fachadas pueden trabajar con varios subsistemas al mismo tiempo.
 */
class Subsystem2
{
    public function operation1(): string
    {
        return "Subsystema2: Preparándome!<br>";
    }
    // ...
    public function operationZ(): string
    {
        return "Subsystema2: Fuego!<br>";
    }
}

/**
 * El código del cliente trabaja con subsistemas complejos a través de una interfaz sencilla
 * proporcionada por la fachada. Cuando una fachada gestiona el ciclo de vida del subsistema,
 * el cliente puede ni siquiera saber de la existencia del subsistema. Este
 * enfoque permite mantener la complejidad bajo control.
 */
function clientCode(Facade $facade)
{
    // ...

    echo $facade->operation();

    // ...
}

/**
 * El código del cliente puede tener algunos de los objetos del subsistema ya creados. En
 * este caso, podría valer la pena inicializar la Fachada con estos objetos
 * en lugar de dejar que la Fachada cree nuevas instancias.
 */
$ subsystem1 = new Subsystem1();
$ subsystem2 = new Subsystem2();
$ facade = new Facade($ subsystem1, $ subsystem2);
clientCode($ facade);

?>
```